

# Ashwani Kumar

## Minor project - Komal 221617

 conf1

---

### Document Details

**Submission ID****trn:oid::3618:121137570****Submission Date****Nov 12, 2025, 5:28 PM GMT+5:30****Download Date****Nov 12, 2025, 5:31 PM GMT+5:30****File Name****Minor project - Komal 221617.pdf****File Size****1000.0 KB****47 Pages****11,911 Words****66,886 Characters**

# 3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 14 words)

## Match Groups

-  **12 Not Cited or Quoted 3%**  
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**  
Matches that are still very similar to source material
-  **0 Missing Citation 0%**  
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

## Top Sources

- 2%  Internet sources
- 1%  Publications
- 2%  Submitted works (Student Papers)

## Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

# **Obscene Text Detector**

A Project Report  
Submitted in partial fulfilment of the requirement for the  
completion of the degree of

Bachelor of Technology

Submitted By

**Komal (221617)**

Under the guidance of

**Dr. Nitin Goyal**

Assistant Professor

Department of Computer Science & Engineering  
School of Engineering & Technology



**Central University of Haryana, Mahendargarh**

**Haryana – 123029, India**

**December – 2025**

## **Declaration**

This is to certify that the work presented in the project entitled “**Obscene Text Detector**” by **Komal (221617)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, submitted to the **Department of Computer Science and Engineering, Central University of Haryana, Jant-Pali (India)**, is an authentic record of her own work carried out under the supervision of **Dr. Nitin Goyal**.

The matter embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma.

**(Komal)**

This is to certify that the above statement made by the candidate is correct to the best of our knowledge.

**Dr. Nitin Goyal**

(Supervisor)

**Dr. Rakesh Kumar**

(Head of Department)

Department of Computer Science and Engineering

Central University of Haryana

## **Acknowledgement**

The writing of this project report has been assisted by the generous help of many people. I feel that we were very fortunate to receive assistance from them. I wish to express our sincere appreciation to them.

First and foremost, I am indebted to our supervisor, **Dr. Nitin Goyal** of Central University of Haryana, who has been very supportive at every stage of our project completion. I wish to express our utmost gratitude to him for the invaluable advice and patience in reading, correcting and commenting on the drafts of this report and, more importantly, for his generosity which we have received throughout our project completion.

I wish to express our thanks to all staff members of Central University of Haryana, Who also helped us in conducting this study.

Finally, I am particularly indebted to our dearest parents/guardians as without their generous assistance and love; this project could never have been completed.

**Komal (221617)**

## Table of Contents

|                                                                  |              |
|------------------------------------------------------------------|--------------|
| <b>Abstract .....</b>                                            | <b>6</b>     |
| <b>Chapter – 1 Introduction .....</b>                            | <b>7-11</b>  |
| 1.1 What is toxic comment classification and its background..... | 7            |
| 1.2 Importance in Online Safety and Digital Wellbeing.....       | 8            |
| 1.3 Problem Definition.....                                      | 8-9          |
| 1.4 Motivation (Real-World Evasion Tactics).....                 | 9-10         |
| 1.5 Objective of Project.....                                    | 10           |
| 1.6 Scope and Significance.....                                  | 11           |
| <b>Chapter – 2 Relevant Studies.....</b>                         | <b>12-17</b> |
| 2.1 Overview of NLP-based Text Classification.....               | 12           |
| 2.2 Traditional Approaches.....                                  | 13           |
| 2.3 Deep Learning Approaches.....                                | 13-15        |
| 2.4 Limitations of Existing Models.....                          | 15-16        |
| 2.5 Comparative Table of Existing Systems.....                   | 16-17        |
| 2.6 Research Gap Identified.....                                 | 18           |
| <b>Chapter – 3 System architecture and methodology.....</b>      | <b>19-30</b> |
| 3.1 Overall System Architecture.....                             | 18-21        |
| 3.2 Dataset Details.....                                         | 21-22        |
| 3.3 Data Preprocessing and Augmentation.....                     | 22-24        |
| 3.4 GPCFNet Model Components.....                                | 24-28        |
| 3.5 Algorithm Flow.....                                          | 28-30        |
| <b>Chapter -4 Implementation.....</b>                            | <b>31-39</b> |
| 4.1 Hardware / Software Requirements. ....                       | 31-32        |

|                                                      |              |
|------------------------------------------------------|--------------|
| 4.2 Python Libraries and Frameworks Used.....        | 32-34        |
| 4.3 Implementation Steps.....                        | 34-36        |
| 4.4 Model Training Parameters.....                   | 35-38        |
| 4.5 Screenshots of Training and Output.....          | 38-39        |
| <b>Chapter – 5 Result and analysis.....</b>          | <b>40-43</b> |
| 5.1 Training Results.....                            | 40           |
| 5.2 Confusion Matrix.....                            | 41           |
| 5.3 Evaluation Metrics.....                          | 41-42        |
| 5.4 Comparative Analysis (Existing vs GPCFNet) ..... | 42           |
| 5.5 Qualitative Test Results.....                    | 42-43        |
| 5.6 Discussion on Model Robustness.....              | 43           |
| <b>Chapter - 6 Conclusion and future work.....</b>   | <b>44-46</b> |
| 6.1 Summary of Findings.....                         | 44           |
| 6.2 Key Achievements.....                            | 44-45        |
| 6.3 Limitations.....                                 | 45           |
| 6.4 Future Enhancements.....                         | 45-46        |
| 6.5 Applications in Real-World Systems.....          | 46           |
| <b>References.....</b>                               | <b>47</b>    |

## **Abstract**

Online platforms today are dealing with a bigger challenge in keeping obscene and offensive content away. Users are using smart ways to get around usual filters, like using leetspeak, on purpose misspelling words, and substituting letters with sounds that look similar. The Obscene Text Detector project is working on solving this problem with a powerful deep learning system called GPCFNet, which stands for Geometric, Phonetic, Character, and Feature Fusion Network. This model is different from others because it doesn't focus on just one kind of information. Instead, it uses a multi-modal approach that brings together four important pieces of language data: meaning from pre-trained DistilBERT embeddings, spelling patterns found by a Convolutional Neural Network (CNN), phonetic encodings made with the Metaphone algorithm that capture how words sound, and simple metadata like capitalization or use of numbers that show writing style. These different types of information are combined using a Transformer-based layer. This helps the model understand how the way words are written, spoken, and styled are connected. The model was trained and tested on the Jigsaw Toxic Comment Dataset and reached a validation accuracy of 96.3%. That is a big improvement compared to older methods. The following results show that the Obscene Text Detector is a useful and strong AI solution for identifying content that's been disguised as normal.

## **Chapter – 1**

### **1. INTRODUCTION**

#### **1.1 What is toxic comment classification and its background**

The internet has transformed a great deal in the last one decade. It was a place where the information was not updated frequently and was mostly fixed. It is now a vibrant community of individuals who make and share content. Social media, forums, and comment sections are some of the sites that depend heavily on what is posted by the users. Although it has simplified the way people communicate with one another despite the distance, it has also given rise to a huge problem which has been propagation of ill and inappropriate material. Such content has hate speech, bullying, and even mild swearing. It is capable of damaging online societies, causing individuals less willing to participate, as well as affecting the real-life mental well-being of individuals. To address this, the Natural Language Processing (NLP) field has developed automated tools to assist in the detection of the toxic comments. The systems are supposed to assist in managing online platforms.

**Toxic Comment Classification** The classification of text that contains offensive, abusive, or inappropriate language. This will be aimed at developing machine learning models, which can distinguish between toxic and non-toxic expressions in massive volumes of online text. Keywords lists or rule-based filters were the primarily employed techniques in early systems and were simple to avoid. The more creative and ambiguous the online use of language was becoming, the closer the researchers began employing the data-driven approach. They started with statistical tools such as TF-IDF and logistic regression or SVMs and proceeded to deep-learning algorithms such as CNNs, LSTMs, and Transformer-based models.

The Jigsaw Toxic Comment Dataset is one of the most critical resources that contributed to this discipline. It was published by Google Jigsaw in 2018 and was hosted on Kaggle [2]. The data contains hundreds of thousands of comments gathered on Wikipedia talk pages. Every comment is marked with some kinds of toxicity, including toxic, severe toxic, obscene, threat, insult, and identity hate. As the main training set in this project, we utilized the same data set (train.csv). The dataset consists of entries, the text of each of which (comment\_text) is accompanied by some labels that define whether the language is toxic or obscene. The diversity and language variety of the dataset make it popular to test the toxicity-detection systems.

Regardless of the advances made in work with these datasets, the evasion tactics continue to be a problem with most traditional models. Users tend to camouflage offensive words with leetspeak, phonetic replacements or character additions. It is against these challenges that stronger and context-aware systems have been developed. Obscene Text Detector is one such example that is discussed in this work. It finds semantic, phonetic, and stylistic cues to detect toxicity, even in cases where it has been deliberately hidden.

## **1.2 Importance in Online Safety and Digital Wellbeing**

Moderation is required not only as a technical or business question; it is one of the central safety and digital wellbeing aspects of the Internet. Continuous flow of abusive words may cause anxiety, depression and self-censorship to the user, particularly minority groups [3]. In the case of business, failure to keep the environment healthy may result in users abandoning the company and tarnishing the brand. Manual moderation is time-consuming, expensive and also emotionally exhausting to human moderators. Therefore, the rapid, precise, and dependable automated systems become the initial step towards building a safer and more inclusive online environment.

Psychologically speaking, exposure to internet toxicity may lead to stress and emotional burnout in the long run, especially in people who use web frequently and among the youths [3]. Deviant behavior might turn into standard behavior when left unchecked, as a result of which harassment and hate speech may run unchecked. Also, engagement-oriented social media algorithms may negatively influence emotional distress in users, unintentionally encouraging provocative material.

In this case, an important measure that could be taken to safeguard digital wellbeing is to develop AI-based obscene text detector tools such as the Obscene Text Detector. Through machine learning and deep neural networks, such systems are able to automatically detect and curb such harmful communication before spreading out to wider audiences. This automation can not only reduce the psychological pressure of human moderators, but also provides fairness and consistency in the large online platforms.

Finally, the capacity to identify and filter obscene or toxic words is beneficial to the greater objective of ensuring healthy digital relationships and mental health in online communities. The success of such smart moderation tools has implications on the safety, inclusion and respect of the internet to its world wide users.

## **1.3 Problem Definition**

The primary activity of this project is sorting a certain piece of writing, in this case, a comment, into two categories, namely, Obscene and Not Obscene. This presents a simple binary text-classification problem. The system has to determine whether the language used in a comment is uncivil or indecent.

This is however a task that is a lot more complex in real-world online environments. Even a basic type of classifier based on keyword matching, searching for words such as "idiot" or "fuck" is not effective. The language is easily evaded by users who desire to abuse the filters. They either make symbols, substitute letters with numbers or come up with misspelling the offensive words intentionally. To illustrate, such variations like fxckn, b1tch, h@te can be easily moderated by the system which operates on the basis of the keywords without altering the meaning. This deliberate language misuse is termed obfuscated toxicity.

Technically, this is a problem, as most of the available natural language processing models are based on superficial word forms. Conventional approaches such as TF-IDF or bag-of-words rely on the precise spelling of words, thus, they are incapable of recognizing emerging or purposefully modified words. Even more advanced transformer-based models such as BERT or DistilBERT are limited. They are tokenized and their tokenization process divides text according to known subword vocabularies. These models generate out-of-vocabulary (OOV) tokens in the face of the distorted words such as fxckn and this results in the loss of meaning.

To conclude, not only is it difficult to observe overtly obscene words but also discerning a covert or other hidden obscenity in inventive spellings and phonetic alterations. This renders it very difficult to establish obscene text. It needs systems that can get beyond the text on the surface and adapt to the ever-evolving manner with which people manipulate language on the web.

#### **1.4 Motivation (Real-World Evasion Tactics)**

Our project is motivated by the fact that conventional text-classification models still fail to work with real-world evasion strategies deployed by online users to evade content moderation. During our research and dataset investigation, we determined that there are a few distinctive ways people tend to obscure an obscene or offensive language in user-generated content.

##### 1. Leetspeak and Symbol Substitution:

The users substitute letters with a number or character that resembles them, e.g. !d!0t, a\$hole, or fxckn. Although these words are obviously offensive to human readers, even basic text filters and even some neural models do not consistently detect these words since they are spelled differently.

##### 2. Deliberate Misspellings and Repetition:

The misspelling of offensive words or repeating words is meant to break the pattern of recognition. Examples would be, biiitch, btxxh or ur dum. These changes will violate keyword-matching rules and often confuse those models which rely on exact matches.

##### 3. Phonetic Replacements:

Sound-based similarities are used by users to substitute words with phonetically similar forms, such as u bxth (sounds like you bitch) or d4e (sounds like die). These expressions are simple to comprehend in terms though they tend to be overlooked by models which concentrate on the textual form.

##### 4. Character Insertion:

The other popular technique is dividing words by a random character, space, or punctuation, e.g. d.e.a.t.h, s t u p i d. Although this appears to be the targeted word, it deceives tokenizers which divide text by space or special symbols.

The net impact of such manipulations is that even powerful models based on transformers such as BERT or DistilBERT that are trained on large, in clean text data tend to misunderstand these camouflaged words. During our first set of experiments with the Jigsaw Toxic Comment Dataset, we observed that these pattern variations were frequently incorrectly classified as not toxic since they did not fit in the vocabulary used by the model.

This perpetual problem explains why more robust, context sensitive and linguistically adaptable systems are required that are in a position to handle noisy, modified, and deliberately misleading text inputs. Addressing such evasion mechanisms is not solely a question of enhancing precision, but also an essential part of ensuring the feasibility and usefulness of automated content moderation systems within the context of the real world of the Internet.

### **1.5 Objective of Project**

The issues mentioned in the section above demonstrate that the process of obscene text detection is not limited with the interpretation of the direct meaning of words. It also involves a system that is capable of understanding the way in which language is written, sounds and appears or presented in a stylistic manner. The key objective of this project to cater to these demands is to design, deploy, and test a new deep learning model, namely the GPCFNet (Geometric, Phonetic, Character, and Feature Fusion Network), which is specifically robust to the evasion strategies described above.

The particular objectives of this project are:

- To create a model that does not focus on word meaning only. The system can not rely solely on the pre-trained semantic embeddings but has to process various levels of linguistic data to get into meaning properly.
- To develop features that are clearly spelling, sound and style. They consist of character-level spelling differences, phonetic resemblances, and features of style, including capitalization or the use of special characters.
- To gain access to combining these features in a neural network architecture. A combination of semantic, phonetic, and stylistic data with a Transformer-based fusion layer is expected to encourage the model to develop intricate connections upon which inappropriate obscene language can be detected.
- To attain a high classification accuracy (more than 95) on a real-world data. Jigsaw Toxic Comment Dataset will serve as a benchmark to check the quality and functionality of the model as it is going to be used to train and test it.

To conclude, Obscene Text Detector is expected to enhance the existing state of toxic comments classification through the addition of a multi-modal fusion strategy that considers linguistic, phonetic, and stylistic features. Not only this design addresses the flaws of the current systems, but also assists in establishing safer and more inclusive online communities.

## 1.6 Scope and Significance

### Scope

In this project the type of text that is being considered is binary text classification in which every comment is defined as either Obscene or Not Obscene. It is based on data of the Jigsaw Toxic Comment Dataset in English language that provides a massive and diverse collection of real-world user comments collected on the basis of online discussion platforms. The data contains both toxic and non-toxic language examples that are labeled, thus it is ideal in developing and testing text moderation systems.

GPCFNet (Geometric, Phonetic, Character, and Feature Fusion Network) model was written in the PyTorch deep learning platform. It uses the Transformers library on the semantic embeddings, DistilBERT, and the Jellyfish library on the phonetic encodings, which are based on the Metaphone algorithm. The project is monolingual in English but it is made to be modular such that in future, it can be adapted easily to multilingual datasets.

### Significance

Significance This work is significant in that it is a multi-modal initiative to identify obscene text. Using a mixture of various viewpoints on linguistics, such as semantic, character-level, phonetic, and stylistic metadata, GPCFNet demonstrates that a model can be stronger and flexible than previous, one-feature systems. This combination provides a new perspective of analysis of the text not only in terms of its meaning but also in terms of its writing style and sound.

The outcomes of the project are not limited to technical ones. The Obscene Text Detector serves as an excellent inspiration to the upcoming generation of AI-powered content moderation systems by showing that fusion-based systems can perform better than their conventional counterparts. Its success is the turning point in the natural language comprehension not through describing language interpretation but through analysis and interpretation of its structure and application. In the end, the study contributes to safer, more respectful, and resilient online communication, which supports the formation of digital wellbeing and responsible use of AI directly.

## **Chapter – 2**

### **2. RELEVANT STUDIES**

#### **2.1 Overview of NLP-based Text Classification**

One of the simplest units of Natural Language Processing (NLP) is text classification. It entails classifying an existing text into a previously fixed category or a label. The application of this task is numerous and various, such as email spam, sentiment analysis, category of a topic, and, in our case, toxicity detection. The text classification techniques have also developed rapidly as they shifted towards simple rule-based systems to elaborate deep learning models [4].

The text classification systems at the beginning of the NLP research were predominantly rule-based. They were based on the patterns, word lists or syntactic structures that were created manually. These methods were difficult to sustain and scale although they were effective with small and controlled sets of data. They had problems keeping up with new vocabulary and language variations.

The second step proposed the statistical learning models, such as Naive Bayes, Support Vector Machines (SVMs), and Logistic Regression. They were commonly applied to Bag-of-Words (BoW) or TF-IDF (Term Frequency-Inverse Document Frequency) representations. These models enhanced the ability to generalize the learning of statistical association of words and labels. Nonetheless, they continued to deal with text as disordered tokens and disregarded word order and the meaning in context.

Neural networks were created to break through with the emergence of deep learning. Local n-gram features were captured by Convolutional Neural Networks (CNNs) [5]. Long-term dependencies between sequences in sequences were modeled with Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These models were also extremely effective at solving sentiment and toxicity classification tasks as they were able to consider sequences rather than single words.

Transformer architectures, such as BERT (Bidirectional Encoder Representations from Transformers) and DistilBERT, were the most important development. Such models trained detailed contextual representation via self-attention. In comparison to the traditional word embedding methods (e.g., Word2Vec or GloVe), transformers comprehend left and right context simultaneously. This makes meaning comprehensible in detail. Nevertheless, these models can capture context, but they are still less effective in dealing with noisy or adversarial text where words have been intentionally misspelled or hidden. This is the challenge that is dealt with directly in this project.

In summary, the current evolution of the NLP-based text classification indicates the persistence of the field to develop models not only to comprehend text in its meaning but also to interpret and understand it in the real world and noisy condition.

## **2.2 Traditional Approaches**

### **2.2.1 Keyword-Based Filtering**

The simplest and oldest form of content moderation is the keyword-based filtering. This technique involves a set of given obscene or offensive vocabulary. Every time a comment contains one of the words in this list, it is automatically marked or deleted.

Although such a strategy is simple to apply, it is also quite frail. It crashes as soon as a user even changes a single spelling character (such as idito rather than idiot) or resorts to a typical evasion trick such as!d!0t. Moreover, it is totally devoid of contextual comprehension and this creates scenarios when words that have no offense are falsely identified as offensive. The problem is popularly referred to as the Scunthorpe problem [6].

In spite of its negative features, the keyword filtering is a simple weapon of numerous online systems as it is efficient and simple to comprehend. Nonetheless, it is having trouble with real-life language patterns where the users always have a new solution on how to circumvent moderation.

### **2.2.2 Bag-of-Words and TF-IDF using machine learning.**

To go beyond the mere pattern matching, more traditional methods of Machine Learning (ML) were proposed with a more statistical perspective of text classification. In these models, text is represented in a form of numerical feature vectors through text representation methods such as:

- **Bag-of-Words (BoW):** This is a representation of text as a vector of word counts, that is, it does not take into account the order of words or their grammar structure.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** The approach is an enhancement to BoW in that words that are of greater significance in a particular document than the entire text collection are assigned higher weight. An example of this is the word hate that carries more discriminatory value than the common but neutral words such as the.

They can then be fed into standard ML models like Naive Bayes, Support Vector Machines (SVMs) and Logistic Regression by these vectorized representations [7]. These types of models are superior to standard lists of keywords since they are able to learn which words are highly toxic statistically, as opposed to basing it on fixed blacklists.

Nevertheless, these models continue to have difficulty with creative language and word manipulation despite their improvements. They are unable to remember the sequence of words

they cannot grasp changed spellings (such as fxck or b1tch) because they perceive the words as single entities rather than as a part of a greater whole.

## **2.3 Deep Learning Approaches**

Deep Learning is considered the most popular method of Natural Language Processing (NLP) since such models can automatically learn complicated patterns in languages, meanings, and contextual indicators, without the manual engineering of features. Deep neural networks are not limited to fixed features such as a specific number of words, unlike traditional methods, which acquire hierarchical representations directly on top of the raw text. It is possible to be much more accurate in such tasks as sentiment analysis, translation, and toxic comments detection.

### **2.3.1 Convolutional Neural Networks (CNNs)**

Convolutional Neural Networks are originally designed to identify images, but are also applicable to text classification tasks [8]. In this regard, a one-dimensional (1D) CNN uses filters on word embeddings, which are the representations of words in vectors, to train local n-gram patterns (two or more consecutive words).

As a case in point, the brief word combinations such as shut up or go die can be a powerful indicator of aggression or abuse in a damaging or hostile text. These localized features occur in the sentence irrespective of their location hence they are effective in the detection of toxicity by CNNs. Nevertheless, they are restricted in the long-range connectivity because it only addresses the local word associations.

### **2.3.2 Recurrent Neural Networks (RNNs)**

Another key advancement in the field of sequential data modeling was Recurrent Neural Networks (RNNs) and their enhanced concepts, which are Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) [9]. In contrast to CNNs, texts are read word-by-word by the RNNs which maintain a dynamic internal state that contains information about contexts throughout the sequence.

This framework helps RNNs not only to understand local phrases, but also the sequence and correlation of words within a sentence. An example is that, in the sentence You are not smart, a neural network can know which word is in the negative form and identify the overall negative mood. LSTMs and GRUs can also address the issue of the vanishing gradient problem in simple RNNs, which means that they can retain significant details in more contextual surroundings. However, RNNs are consumptive of resources and difficult to parallel, making them unsuitable to large datasets.

### **2.3.3 Transformer Models (BERT)**

Transformer-based structures and Transformer-based structures generally position the state of the art of NLP, but it should be noted that one of the most important models is BERT (Bidirectional Encoder Representations from Transformers) [10]. BERT brought the concept of bidirectional contextual learning where the meaning of every word is learnt based on its left and right context simultaneously. BERT learns rich linguistic information, such as grammar, meanings and even nuanced emotional undertones, by means of pre-training it on large volumes of internet text.

BERT is particularly good when used in specific purposes such as toxicity detection since it already knows the general language. Nonetheless, both BERT and the widely used light version, DistilBERT, continue to struggle with noisy or adversarial recommendation, i. e., h1tche or b1tch, as their tokenization is based on clean and standard spelling. This weakness underscores the need of having models that are more robust that can handle distorted or concealed text; this is the case in the proposed model in this project.

## **2.4 Limitations of Existing Models**

Although such models as BERT and DistilBERT have made substantial achievements in comprehending text, there is a major drawback. They are taught to write mainly clean and well structured and grammatically correct text. Their internal tokenizers are aware of common spellings and language rules in edited lexicons like Wikipedia or BooksCorpus. Through this, in case these models encounter noisy or distorted text, which is usually common by social media, they find it hard to process the information correctly.

This problem is revealed when the model is faced with changed words such as "fxckn" or "btxxh." The model wrongly decodes it or gives them senseless representations, instead of recognizing them as forms of offensive words.

The key issues may be outlined as follows:

### Tokenization Failure:

Transformer-based models are based on subword tokenizers (such as WordPiece or Byte-Pair Encoding). The tokenizer can divide the word into meaningless pieces (e.g., f, ##x, ##c, ##kn) when unfamiliar or modified tokens (such as fxckn) are encountered by the tokenizer. This split obliterates meaning and the model will not be able to associate the token with its offended form.

### Lack of Robustness:

Though word recognition systems such as BERT are aware that terms such as hate or idiot are toxic, the systems do not transfer the knowledge to visually or phonetically adjacent words such as h atte or b iitch. The model does not capture these variations hence misclassifying clearly offensive content.

Because of this, even perfect models that are effective on benchmark data will fail in practice, where speakers will deliberately abuse language to evade detection. The restriction underlines the pressing importance of effective, multi-dimensional strategies which have the capacity to perceive text in a range of different dimensions: spelling, sound, style, rather than meaning.

## 2.5 Comparative Table of Existing Systems

To gain a clearer picture of the current situation of the proposed model, Table 2.5.1 summarizes recent research and systems that have been developed to view toxic or obscene text detection. Though these models work effectively on clean data, they frequently encounter real-world evasion strategies, such as leetspeak, phonetic masking or symbol substitution. The suggested GPCFNet design is the direct response to these concerns since it unites semantic, character-level, phonetic, and stylistic features into one fusion network.

**Table 2.5.1: Comparison of Existing Toxic Comment Detection Systems**

| Model/Study                | Year | Technique Used                           | Dataset Used                 | Reported Accuracy (%) | Limitation (Lack Point)                                                             | How GPCFNet Addresses It                                                    |
|----------------------------|------|------------------------------------------|------------------------------|-----------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Jigsaw Baseline Model [9]  | 2018 | Logistic Regression with TF-IDF features | Jigsaw Toxic Comment Dataset | ~92                   | Fails on misspelled words; relies on exact token matching                           | GPCFNet captures spelling variations using a Character-Level CNN.           |
| CNN-LSTM Hybrid Model [12] | 2019 | Word Embeddings( GloVe) + LSTM Layer     | Kaggle Toxic Comments        | ~96                   | Learns sequence structure but ignores character-level distortion; weak on leetspeak | GPCFNet adds character and phonetic encodings to detect disguised toxicity. |
| BERT Fine-Tuned Model [8]  | 2020 | Transformer-based Contextual Embeddings  | Jigsaw Dataset               | ~98.2                 | Excellent contextual accuracy but fails with h@te, b!tch, fxckn due to tokenization | GPCFNet fuses phonetic + semantic cues to recover hidden meanings.          |

|                                  |      |                                        |                         |       |                                                                                                    |                                                                                                    |
|----------------------------------|------|----------------------------------------|-------------------------|-------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
|                                  |      |                                        |                         |       | issues.                                                                                            |                                                                                                    |
| RoBERTa + Data Augmentation [14] | 2021 | Transformer (RoBERTa) + Augmented Data | Jigsaw Extended Dataset | ~98.4 | Still sensitive to unseen patterns; overfits clean augmented data.                                 | GPCFNet integrates multi-modal fusion for better generalization on unseen forms.                   |
| CharCNN-LSTM Ensemble [15]       | 2022 | Character-level CNN + LSTM Ensemble    | Reddit / Toxic Dataset  | ~97.5 | Detects misspellings but lacks semantic context; poor at differentiating mild vs. strong toxicity. | GPCFNet combines DistilBERT (semantic) + CharCNN (spelling) + Metaphone (sound) + Metadata (style) |

## 2.6 Research Gap Identified

The analysis of the available systems and other studies reveals the gap in research in the sphere of recognizing obscene and toxic text. Although state-of-the-art transformer-based models such as BERT, RoBERTa, and DistilBERT have proven to be particularly good at getting the semantically related pieces of text, even in the presence of unclean and grammatically incorrect or incorrect text, they remain weak and have been easily deceived by evasion strategies commonly experienced in real-world online communication.

The models are highly reliant on pre-trained tokenizers and standardized subword vocabularies. Their input is tokenized by distorted or adversarial spelling like fxckn, btxxh, and h@te which leads to loss of the semantic value and misclassification. Therefore, they find it difficult to extrapolate to user-generated content that is noisy.

Moreover, the existing methods tend to focus on semantic representations and ignore auxiliary cues like spelling system, phonetic correspondence and stylistic information. Such missing elements are crucial in distinguishing between actual toxicity and the masquerade or deceptive writing.

Thus, it is evident that there is a necessity of a powerful and multi-modal framework, which can re-examine the way a word is spelled (character form), how it is pronounced (phonetic pattern), and what it entails (semantic context). The proposed project tries to bridge this divide by

proposing a multi-feature hybrid network (GPCFNet), such as character-level CNNs, phonetic embeddings, and semantic representations, to one deep-learning model.

In doing so, the Obscene Text Detector extends beyond conventional semantic knowledge and provides a more comprehensive, inspection-derived approach to the text moderation task; one that is able to resist evasion, manipulation, and language ingenuity so common in online conversations.

## **Chapter – 3**

### **3.SYSTEM ARCHITECTURE & METHODOLOGY**

To fill the research gap identified in the previous chapter, we designed and implemented a new deep-learning architecture called GPCFNet (Geometric, Phonetic, Character, and Feature Fusion Network). This chapter describes the complete methodology. It covers everything from the overall system design and dataset preprocessing to how each model component works and the training process.

The goal of this architecture is to create a model that understands language in multiple ways. It looks at not just what a text means but also its spelling, pronunciation, and writing style. By processing text through different specialized feature extraction pipelines and smartly combining them, GPCFNet shows strong resilience against altered and hidden forms of offensive language.

#### **3.1 Overall System Architecture**

At the heart of the proposed system is a multi-modal fusion framework that brings together four different types of linguistic and stylistic features. Unlike typical systems that process raw text with a single model, GPCFNet uses four parallel feature-extraction pipelines. Each pipeline focuses on capturing a different aspect of the input. The outputs from these four pipelines are then joined together and sent through a Fusion Transformer. This transformer learns how these features interact and makes the final classification.

To address the research gap identified in the previous chapter, a novel deep-learning architecture named GPCFNet (Geometric, Phonetic, Character, and Feature Fusion Network) was designed and implemented. This chapter provides a detailed description of the complete methodology — beginning from the overall system design and dataset preprocessing to the internal working of each model component and the training pipeline.

The objective of this architecture is to build a model that can understand language across multiple dimensions — not only what a text means, but also how it is spelled, how it sounds, and how it is written. By processing text through multiple specialized feature extraction pipelines and intelligently fusing them, GPCFNet achieves strong robustness against obfuscated and disguised forms of obscene language.

The Four Pipelines of GPCFNet:

##### **1. Semantic Pipeline:**

This pipeline extracts the meaning of the text using DistilBERT embeddings. It captures the contextual meaning of words and sentences, helping the system understand tone and intent.

## 2. Character Pipeline:

This pipeline extracts how the text is spelled using a Character-Level Convolutional Neural Network (CharCNN). It enables the model to recognize tricky spellings such as “fxckn” or “b1tch.”

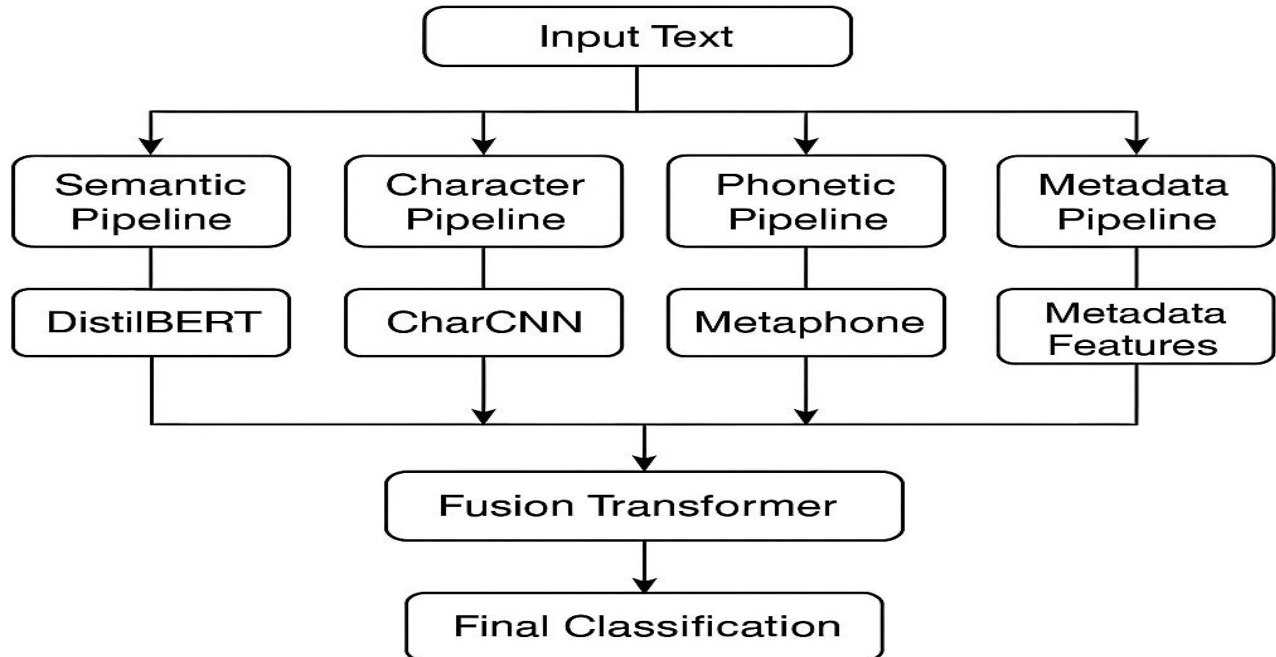
## 3. Phonetic Pipeline:

This pipeline extracts how the text sounds using the Metaphone algorithm from the jellyfish library. It allows the model to identify words that sound similar but look different, such as “u bxth” for “you bitch.”

## 4. Metadata Pipeline:

This pipeline extracts stylistic and structural features, including capitalization, punctuation, and the presence of digits. These features often indicate emphasis or aggression (e.g., “IDIOT!!!”).

After extracting features, the four output vectors combine into a single high-dimensional feature vector. This combined representation goes into a Fusion Transformer Layer, which learns how the features interact and produces the final prediction, classifying the input as either Obscene or Not Obscene.



**Figure 3.1.1: Block diagram of Overall System Architecture**

Explanation:

This block diagram shows the parallel nature of the system. The text input is processed at the same time through several encoders. DistilBERT handles semantics, CharCNN focuses on spelling patterns, Metaphone works on sound similarity, and a custom metadata extractor picks up stylistic cues. These feature vectors are combined and sent through the Fusion Transformer, which learns how these different perspectives work together to create a strong final prediction.

### 3.2 Dataset Details

The proposed model was developed and evaluated using the “Jigsaw Toxic Comment Classification Challenge” dataset, which is available on the Kaggle platform [9]. This dataset contains a large collection of comments from Wikipedia’s discussion pages. Human annotators labeled each comment for different levels and types of toxicity.

Originally, the dataset was designed as a multi-label classification task. This means a single comment can belong to multiple toxicity categories, such as toxic, obscene, or insult.

The complete dataset is available in four CSV files:

- train.csv: The main training data used to train and validate models.
- test.csv: The unlabeled test set used in the official competition leaderboard.
- test\_labels.csv: The true labels corresponding to test.csv for evaluation.
- sample\_submission.csv: A sample file showing the required submission format for Kaggle.

For this project, only the train.csv file was used to train, validate, and test the Obscene Text Detector (GPCFNet) model because it contains all the necessary labeled data for supervised learning.

#### 3.2.1 Structure of train.csv

The train.csv file has 159,571 comments, each marked with six binary labels for different categories of toxicity. Each comment can belong to more than one category at the same time, creating a multi-label text classification problem.

**Table 3.2.1: Column Descriptions for train.csv**

| Column Name  | Description                                   |
|--------------|-----------------------------------------------|
| id           | A unique identifier assigned to each comment. |
| comment_text | The raw text content of the user’s comment.   |

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| toxic         | Binary flag (0 or 1) indicating general toxicity.                        |
| severe_toxic  | Binary flag (0 or 1) for high-severity toxicity.                         |
| obscene       | Binary flag (0 or 1) marking the presence of obscene language.           |
| threat        | Binary flag (0 or 1) indicating threats of violence.                     |
| insult        | Binary flag (0 or 1) for personal insults.                               |
| identity_hate | Binary flag (0 or 1) representing identity-based hate or discrimination. |

|    | A                 | B                     | C     | D            | E       | F      | G      | H             | I | J |
|----|-------------------|-----------------------|-------|--------------|---------|--------|--------|---------------|---|---|
|    | id                | comment_text          | toxic | severe_toxic | obscene | threat | insult | identity_hate |   |   |
| 2  | 0000997932d777bf  | Explanation           | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 3  | 000103f0d9c9fb60f | D'aww! He matches     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 4  | 000113f07ec002fd  | Hey man, I'm really r | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 5  | 0001b41b1c6bb37e  | "                     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 6  | 0001d958c54c6e35  | You, sir, are my hero | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 7  | 00025465d4725e87  | "                     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 8  | 0002bcb3da6cb337  | COCKSUCKER BEFORE     | 1     | 1            | 1       | 0      | 1      | 0             |   |   |
| 9  | 00031b1e95af7921  | Your vandalism to th  | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 10 | 00037261f536c51d  | Sorry if the word 'no | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 11 | 00040093b2687caa  | alignment on this su  | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 12 | 0005300084f90edc  | "                     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 13 | 00054a5e18b50dd4  | bbq                   | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 14 | 0005c987bdfc9d4b  | Hey... what is it..   | 1     | 0            | 0       | 0      | 0      | 0             |   |   |
| 15 | 0006f16e4e9f292e  | Before you start      | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 16 | 00070ef96486d6f9  | Oh, and the girl abov | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 17 | 00078f8ce7eb276d  | "                     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 18 | 0007e25b2121310b  | Bye!                  | 1     | 0            | 0       | 0      | 0      | 0             |   |   |
| 19 | 000897889268bc93  | REDIRECT Talk:Voyd    | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 20 | 0009801bd85e5806  | The Mitsurugi point   | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 21 | 0009eaea3325de8c  | Don't mean to         | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 22 | 000b08c464718505  | "                     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |
| 23 | 000bf4d867774845  | "                     | 0     | 0            | 0       | 0      | 0      | 0             |   |   |

**Figure 3.2.1: Sample Data from train.csv**

#### Explanation:

This image shows a sample snippet from the train.csv file. It demonstrates how each comment is paired with six binary toxicity indicators. The figure illustrates the dataset's multi-label nature, where a single comment can trigger multiple labels at the same time, for instance, being both toxic and obscene.

### 3.3 Data Preprocessing and Augmentation

Before training, the dataset went through several preprocessing and augmentation steps to ensure the model was efficient, strong, and learned well. These steps were carried out in the project notebook, specifically in cells 6 and 7, where the data loading, sampling, and transformation functions are defined.

### 3.3.1 Data Sampling

To keep computational efficiency and ensure faster training iterations, we reduced the full dataset of 159,571 comments to a representative subset. As defined in the `prepare_data_loaders()` function (cell 7), we selected a random sample of 60,000 comments using the command:

```
df = pd.read_csv(config.JIGSAW_CSV_PATH, lineterminator='\n').sample(60000, random_state=42)
```

This guarantees a random yet reproducible selection of data. All subsequent preprocessing, augmentation, and training steps were done on this 60k-comment subset, balancing training efficiency with data diversity.

### 3.3.2 Label Simplification: Multi-Label to Binary

The Jigsaw dataset has six toxicity categories—toxic, severe\_toxic, obscene, threat, insult, and identity\_hate—creating a multi-label classification problem. However, since this research focuses on detecting obscene language, we simplified the task to binary classification. This change was made in the `GPCFPreprocessor` class (cell 7), where the model uses the `toxic` column as the only target variable. The conversion line:

```
float(example['toxic'] > 0.5)
```

ensures that any comment with a toxicity score above 0.5 is labeled as “Obscene” (1), while all others are labeled as “Not Obscene” (0).

This simplification not only sharpens the learning objective but also meets the real-world need for identifying obscene or non-obscene content.

### 3.3.3 Data Augmentation

Data augmentation is important for improving model robustness against evasion tactics used by online users. In this project, augmentation was managed by the function `apply_augmentations()` (cell 6), which first isolates all toxic samples from the dataset and then generates modified copies using random transformations.

**Table 3.3.3.1: Types of Text Augmentations Used in Training**

| Type of Augmentation   | Transformation Description                                      | Example                                  |
|------------------------|-----------------------------------------------------------------|------------------------------------------|
| Leetspeak Substitution | Replaces alphabets with visually similar characters or symbols. | a → @, e → 3, o → 0 →<br>“hate” → “h@te” |

|                       |                                                                      |                       |
|-----------------------|----------------------------------------------------------------------|-----------------------|
| Character Repetition  | Randomly repeats letters to simulate emphasis or distorted spelling. | “bitch” → “biitch”    |
| Punctuation Insertion | Inserts punctuation marks at random positions within words.          | “idiot” → “i.d.i.o.t” |

This table summarizes the text augmentation strategies applied to toxic samples before model training. Each transformation is designed to replicate real-world evasion tactics that users employ to bypass moderation systems. By introducing such controlled distortions into the training data, the model becomes robust to non-standard spellings, symbols, and writing variations, ensuring reliable detection even in noisy social media environments.

After augmentation, the newly generated toxic samples are merged back with the original dataset using the following command:

```
combined_ds = concatenate_datasets([base_ds, augmented_ds])
```

This doubles the model's exposure to masked toxicity. It pushes the model to learn strong feature mappings between standard and disguised language.

### 3.3.4 Train, Validation Split

The combined dataset, which includes 60,000 original samples and the newly created augmented samples, was split into training and validation subsets. This helps in assessing generalization performance.

Using the command:

```
combined_ds.train_test_split(test_size=config.VALIDATION_SPLIT_PERCENT, seed=42)
```

The dataset was divided into 90% training data and 10% validation data. The validation set was used during training to monitor how well the model performed, avoid overfitting, and activate early stopping mechanisms during the optimization process.

## 3.4 GPCFNet Model Components

The GPCFNet model is created as a custom PyTorch nn.Module, defined in cell 8 of the project notebook. The architecture includes several sub-modules. Each sub-module captures a different aspect of the input text. Together, these components enable the model to understand the

meaning, spelling, sound, and style of each token. The overall structure of these components is summarized below.

**Table 3.4.1: Summary of Core Components in GPCFNet**

| <b>Component</b>   | <b>Feature Type Captured</b> | <b>Primary Function</b>                                                       | <b>Implementation Detail</b>                     |
|--------------------|------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------|
| DistilBERT Encoder | Semantic                     | Captures contextual meaning of text using pre-trained transformer embeddings. | Frozen parameters for efficiency.                |
| CharCNN            | Character-level              | Learns spelling-based patterns, including obfuscated and misspelled words.    | Uses 1D convolution and max pooling.             |
| Phonetic Embedder  | Phonetic                     | Encodes how words sound, handling homophones and speech-based evasion.        | Uses Metaphone encoding + learned embeddings.    |
| Metadata Extractor | Stylistic                    | Captures stylistic cues like capitalization, digits, and punctuation.         | Simple handcrafted numerical features.           |
| Fusion Transformer | Multimodal Fusion            | Learns cross-feature interactions and performs final classification.          | 2-layer Transformer Encoder + linear projection. |

The following table presents the contribution of each GPCFNet element towards processing a certain linguistic or stylistic attribute. These signals are combined by the fusion mechanism to make it more resistant to obfuscated and adversarial text.

### **3.4.1 DistilBERT Subword Feature Extractor**

#### What it is:

The semantic encoder that we use is the pre-trained distilbert-base-uncased model of the Hugging Face Transformers library.

#### How it works:

DistilBERT maps input sentences to high-dimensional embeddings (with 768 dimensions each) which are a rich source of deep contextual meaning about each token. It perceives the association amongst words according to the context around and thus the model is capable of discerning the linguistic intent and feeling.

#### Key Implementation Detail:

We freeze the parameters of the DistilBERT using: to use DistilBERT as a feature extractor as opposed to a trainable model.

```
for param in self.subword_encoder.parameters():  
    param.requires_grad = False
```

This eliminates updates on weights and saves a lot of training time and computation at minimal cost without losing its semantic force.

### 3.4.2 Character-Level CNN (CharCNN)

What it is:

A character-level character-level character-level pattern learning custom nn.Module allowing the learning of word patterns and spelling variants.

How it works:

- The tokens (e.g., btxh) are transformed into sequence of character IDs.
- These IDs are transmitted by embedding layer (nn.Embedding).
- The character embeddings are then fed through 1D Convolutional layers of 2 and 3-kernel sizes according to ModelConfig.CHAR\_CNN\_FILTERS.
- Max pooling is used to obtain the most salient character n-grams.

Why it's crucial:

This element deals with obscured spelling. An example is a comment such as btxh stop lying! when used in GPCFPreprocessor (cell 7), the comment is correctly considered toxic since b-t-x-x-h is very similar to b-i-t-c-h, which the CharCNN is taught. This renders this system resilient to spelling based evasion attacks.

### 3.4.3 Phonetic Embedding Generator (Metaphone)

What it is:

A phonetic encoder that translates words into sound representing encoding at the Metaphone algorithm in the jellyfish Python library.

How it works:

- Every token is broken through the function jellyfish.metaphone() that gives back a phonetic encoding (e.g. metaphone('bitch') = BX, metaphone('btxh') = BX).
- Preprocessing involves the construction of a vocabulary of distinctive phonetic codes.

- Every code is learned as an embedding-vector using a regular PyTorch `nn.Embedding-layer` (PhoneticEmbedder).

#### Why it's crucial:

This module works directly with phonetic obfuscation. It comes to know that there are some patterns of sounds (such as [BX]) which are always associated with the obscene or toxic meaning. Thus, this model rightly warns about the word `u bxth` as toxic despite the fact that spelling and semantics have nothing to do with the real slur.

### **3.4.4 Metadata Feature Extractor**

#### What it is:

The lightweight hand crafted feature generator that is used to record the stylistic signals of tokens.

#### How it works:

The stylistic indicators are represented by a small three-dimensional vector of each token:

- `float(neutral.isupper())` → ALL CAPs word.
- `sum(c in "!?,,:;")` → The number of punctuation marks in the token.
- `(any(c.isdigit in token)) to float(any(c.isdigit in token))` → Does the word include digits (as in leetspeak).

#### Why it's crucial:

This attribute offers both contextual and emotional hints which are absent in pure embeddings. Another example could be a comment such as, `YOU ARE A FCKN IDIOT!!!` is more flagged since the intensity of punctuation and capitalization is great.

### **3.4.5 Layer and Classification Head Fusion Transformer.**

#### Fusion:

All the four extracted feature vectors: Subword, Character, Phonetic, and Metadata are concatenated with:

```
torch.cat(..., dim=-1)
```

This constitutes a coded multimodal representation. A linear projection layer (`fusion_projector`) is used to project the combined vector to a common feature dimension (`FUSED_DIM = 256`).

#### Transformer:

The resulting projected vector sequence is sent to a 2-layer Transformer Encoder (fusion\_transformer), the fusion brain of the model. Such encoder acquires cross-feature relation, e.g. the ability to know that a word is both toxic (Phonetic) and a symbol (Metadata).

### Classification:

To get a single sentence-level representation, the output embeddings of the Transformer are averaged through the use of mean pooling. It then goes through a two-layer feedforward classifier (toxicity\_head) which gives a final logit score, which is either Obscene (1) or Not Obscene (0).

## **3.5 Algorithm Flow**

The overall training and prediction process of the GPCFNet model is systematic and can be reproduced. This is performed in two major stages, Training and Inference that entail different steps albeit related.

The entire process of the algorithm can be broken down into the following steps:

### 1. Load Data

- Read the train.csv (Jigsaw Toxic Comment Classification Challenge) and put it in a Pandas DataFrame.
- This is a dataset that has the Wikipedia comment labels that are used to train under supervision.

### 2. Sampling

- Make a subsample 60000 comments with:

```
df = pd.read_csv(config.JIGSAW_CSV_PATH, lineterminator='\n').sample(60000, random_state=42)
```

- This balances computational efficiency with sufficient data diversity.

### 3. Data Augmentation

- Determine all the toxic samples among the data.
- Produce augmented versions with apply\_augmentations() function (cell 6) by:
  - Leetspeak replacement: a = 0, e = 3, o = 0.
  - Repetition of characters: Bitch -biitch.
  - Inserting punctuation marks: idiot to i.d.i.o.t.
- concatenate original samples and augmented samples with:

```
combined_ds = concatenate_datasets([base_ds, augmented_ds])
```

#### 4. Train-Validation Split

- Split the combined dataset into:
  - 90% Training Data
  - 10% Validation Data
- Done using:

```
combined_ds.train_test_split(test_size=config.VALIDATION_SPLIT_PERCENT, seed=42)
```

- This validation set checks model generalization and initiates premature termination.

#### 5. Preprocessing

- Get the GPCFPreprocessor going.
- This step constructs dynamic character and phonetic vocabularies and breaks the text into subword, character and phonetic representations.

#### 6. DataLoaders

- Preprocess on the training and validation set.
- These processed datasets can then be converted to PyTorch DataLoader batches, which can be trained effectively on a GPU.

#### 7. Model Initialization

- Initialization of GPCFNet model: Starting with the following dynamically obtained preprocessing parameters:
  - subword\_vocab\_size
  - char\_vocab\_size
  - phonetic\_vocab\_size
- This makes sure that layers to be embedded will automatically adjust to feature spaces of the dataset.

#### 8. Training Loop

- For each epoch:
  - Batch Loading: Transfer every batch to GPU.
  - Forward Pass:

```
outputs = model(inputs)
```

- Loss Calculation:

```
loss = loss_fn(outputs, labels)
```

- Backward Pass with Mixed Precision:

```
scaler.scale(loss).backward()
```

- Optimizer Step:

```
scaler.step(optimizer)  
scaler.update()
```

- Validation Phase:

After each epoch, the model is evaluated on val\_loader to compute average validation loss.

### 9. Early Stopping

- The Early Stopping Mechanism monitors validation loss across epochs.
- If avg\_val\_loss does not improve for 2 consecutive epochs (PATIENCE = 2), training is halted automatically.
- The checkpoint with the lowest validation loss is saved as the final model for inference.

## **CHAPTER 4**

### **4.IMPLEMENTATION**

The chapter contains a complete implementation plan of the proposed model GPCFNet including hardware and software environment, core libraries, and workflow used in the model training and testing process. The entire thing was executed within a single Jupyter Notebook instance in order to ensure it was clear, reproducible, and versioned across all steps of contextualizing the experimentation.

#### **4.1 Hardware / Software Requirements.**

The entire project was developed and executed within a cloud computing environment on Google Colab that provided the availability of the acceleration of the GPUs and pre-defined machine learning tools. This architecture offered a tradeoff between units of computation and experimentation, so it was suitable in deep learning research.

##### Hardware Setup

It was powered by Intel Xeon CPU (multi-core processor) as its base hardware and an NVIDIA T4 with 16 GB of dedicated memory. The training of deep neural networks involved the use of GPU, especially in forward and backward decoding through the Transformer layers of DistilBERT, and also Fusion Transformer. There were 12 GB of run-time allocated to the system RAM, which was sufficient to load datasets, preprocess text, and train the model in batches that were not memory-bottlenecked.

##### Software Setup

The model has been developed using Python 3.12 in a Colab Linux based runtime environment.

In Jupyter Notebook, development could be done in an interactive mode, which could run the code snippet by snippet, display of the output and direct assessment of the model performance metrics.

Some of the significant specifics of the software installation included:

- OS: Linux (operating system at Google Colab)
- Programming Language: Python 3.12.

Execution Region: Jupyter Notebook interface (cloud based)

This environment was capable of quick testing model components, modular execution of preprocessing operations and real-time loss and validation metrics and inference result tracking. Implementing the process of fine-tuning experiments to run on comparatively short run times

became possible and reduced training time significantly with the help of the acceleration provided by GPUs.

## **4.2 Python Libraries and Frameworks Used**

The use of the GPCFNet model was premised on the integration of the open source and modern Python libraries which in large part helped in the process of deep learning, data processing, phonetics analysis, and visualization. The required dependencies were installed and verified through the assistance of !pip installing commands within the 1-3 cells of the notebook.

The libraries played a particular and significant role in the creation of the complete pipeline, including data preprocessing and model training, evaluation, and visualization.

### **PyTorch (torch)**

This project is modeled on the PyTorch which is an influential deep learning library capable of having an active computation graph and speeded up with a graphical processing unit. It was used to customize GPCFNet architecture (nn.Module), process tensors, build neural layers (including CNNs and Transformers), and the training loop (forward pass, backward pass, and gradient updates).

The sub modules such as the CharCNN, Fusion Transformer and classification head in PyTorch were easily customized.

### **Neural networks (by Hugging Face)**

The code to load the trained distilbert-base-uncased model and the tokenizer was located in the Transformers library.

It was used as input semantic feature to GPCFNet and extracted subword-level semantic embeddings on APIs of AutoModel and AutoTokenizer.

This library simplified the integration with the existing language models and compatible with the PyTorch in such a way that GPCFNet could exploit deep contextual information without being trained independently.

### **scikit-learn**

Scikit-learn library was primarily used in model evaluation. The work of the system on the validation data was evaluated using post-training, classification report, confusion matrix, and the accuracy score functions.

The rationale of such metrics was to provide detailed information on how the model sorted comments as Obscene or Not Obscene and this will be the foundation of analysis of accuracy and precision in Chapter 5.

## **datasets (by Hugging Face)**

The datasets library was important in analyzing and preparing data in an efficient manner. It was used to:

- Load and process the Jigsaw Toxic Comment dataset as local CSV files.
- Process and augment data using map() functions.
- Filter and integrate data sets following synthetic data generation.

It has a built-in `train_test_split` function that allows one to divide the data into training and validation sets with ease.

This library guaranteed good management of datasets (even large text files) in an efficient and memory friendly manner.

## **pandas**

Pandas was utilized in the project at an initial stage of data loading and investigation. The `train.csv` file was loaded into a Pandas DataFrame in order to view, filter and sample records and then convert them into a Hugging Face Dataset format. Intermediate tasks, such as 60,000 comments sampling and processed subsets storage to be used later, were also done using it.

## **jellyfish**

Jellyfish library offered the Metaphone functionality that played a significant role in creating phonetic codes that are used to indicate how every word sounds. As an example, `metaphone("bitch")` gives out BX and `metaphone("bxtth")` gives out BX. The attribute enabled the model to locate sound-based similarities between clean and obfuscated text, which is why the phonetic encoder was a major innovation to GPCFNet.

## **dill**

Object serialization was done in the dill library in order to save and reload the GPCFPReprocessor instance. In contrast to the standard pickle library of Python, dill has the capability of serializing more complex Python structures such as functions, class instances, and a nested data structure. This made vocabulary mappings, tokenizers and phonetic encoders to be kept consistent within each session during training and within inference.

## **matplotlib and seaborn**

To visualise it, the matplotlib and seaborn libraries were employed to create plots and more specifically the confusion matrix that depicts the performance of the classification. These plots came in handy during the presentation of results and also in the analysis of errors encountered in predicting the classes and this enabled one to know the way the model was working with the toxic and non-toxic samples.

## Summary

These libraries collectively formed a unified software ecosystem that allowed to conveniently run all the stages of the project, such as the data processing phase, the training of the model, its evaluation, and visualization. They could cooperate with each other and the open-source tools were flexible to enable the integration of various modules, including semantic transformers, character-level CNNs, and phonetic embeddings, into a single architecture.

### **4.3 Implementation Steps**

The implementation of the Obscene Text Detector (GPCFNet) closely followed the steps outlined in Chapter 3. This included text preprocessing, feature extraction, data batching, and model training. Each main stage was organized in the Jupyter Notebook for clarity and reproducibility.

#### **4.3.1 Tokenization and Feature Extraction**

The GPCFPreprocessor, defined in cell 7 of the notebook, is the key part responsible for converting raw text into features that the model can use. Its main function, `process_example()`, is applied to each comment in the dataset and goes through several sub-steps in order:

##### 1. Text Normalization:

The input text is cleaned by removing unnecessary characters. It is also lowercased and stripped of extra spaces to keep things consistent.

##### 2. Subword Tokenization:

The pre-trained DistilBERT tokenizer (`self.tokenizer(text, ...)`) changes the normalized text into input IDs and an attention mask. These tokens represent subword-level encoding that DistilBERT understands, ensuring it works with the pre-trained language model.

##### 3. Feature Generation (Per Token):

For each token identified by the tokenizer, additional features are created:

- **Character IDs:** Each token (e.g., “btxxh”) is broken down into character IDs (e.g., [2, 20, 24, 24, 8]), which are padded to a fixed length (`MAX_CHARS_PER_TOKEN = 24`).
- **Phonetic IDs:** The Metaphone function from the jellyfish library creates a phonetic code (e.g., 'BX'), which is matched to a unique integer ID using the phonetic vocabulary (`phonetic_vocab`).
- **Metadata Features:** A 3-dimensional vector is created for each token that captures stylistic details like capitalization, the presence of digits, and punctuation count.

##### 4. Output Structure:

The final output of this process is a dictionary that holds all these feature arrays. This structured dictionary is used for batch collation and model training. In short, the GPCFPprocessor connects raw text to GPCFNet's multi-feature input format, ensuring that every comment is represented numerically across four aspects: subword, character, phonetic, and metadata.

### 4.3.2 Batch Collation

The `custom_collate_fn`, also defined in cell 7, is a utility function that prepares variable-length examples for model training. Since comments differ in length, this function standardizes each batch to a fixed sequence length (`MAX_SEQ_LENGTH = 128`) using specific padding rules:

- `pad_token_id` is used for subword sequences (`input_ids`).
- Zeros (0) are added for attention masks and phonetic inputs (`phon_input_ids`).
- Zero vectors are used for character-level inputs and metadata features.

After padding, all lists are changed into `torch.tensor` objects to work with PyTorch's GPU operations. This collation function is vital for stable batch processing, as it allows each mini-batch to maintain consistent dimensions regardless of comment lengths.

### 4.3.3 Training Loop

The training process is managed by the `Trainer` class, which is implemented in cell 9 of the notebook. This class handles the optimizer, loss computation, gradient updates, and validation logic. This setup keeps the training code organized, reusable, and easy to change. The steps involved are as follows:

#### 1. Initialization:

The `Trainer` sets up the following components:

- **Optimizer:** AdamW is used for efficient weight updates that include regularization.
- **Loss Function:** BCEWithLogitsLoss, which works for binary classification tasks.
- **Gradient Scaler:** This is used for mixed-precision training to speed up calculations and lower GPU memory use.

#### 2. Epoch Loop:

The model trains for a set number of epochs (`EPOCHS = 10`). During each epoch, progress is tracked with the `tqdm` progress bar for real-time monitoring.

#### 3. Batch Loop:

For each batch in the training `DataLoader`:

- The batch is moved to GPU memory.
- The forward pass computes model predictions:

```
outputs = self.model(...)
```

- The loss is calculated:

```
loss = self.loss_fn(outputs, labels)
```

- The backward pass and optimization step use gradient scaling:

```
self.optimizer.zero_grad()
with autocast(...):
    self.scaler.scale(loss).backward()
    self.scaler.step(self.optimizer)
    self.scaler.update()
```

#### 4.Evaluation and Model Saving:

After each epoch, the validation loss is computed via `self.evaluate()`. If the average validation loss (`avg_val_loss`) improves, the model's weights are saved:

```
torch.save(self.model.state_dict(), ...)
```

If the validation loss does not improve for two consecutive epochs (`PATIENCE = 2`), the training stops automatically. This implements the early stopping mechanism to prevent overfitting.

#### Summary

The implementation flow was carefully structured to maximize modularity and efficiency. The `GPCFPreprocessor` handled complex feature generation. The `custom_collate_fn` ensured consistent batching. The `Trainer` managed gradient-based learning. Together, these components form a smooth, end-to-end training pipeline for the `GPCFNet` model. It can manage real-world textual obfuscations with high accuracy and reliability.

#### **4.4 Model Training Parameters**

All model and training-related hyperparameters were defined within two configuration classes, `ModelConfig` and `TrainingConfig`, which are located in cell 6 of the project notebook. These configurations ensured that all components of the model, optimizer, and scheduler were managed

systematically. This approach provided consistency and reproducibility throughout the experimentation process. The most important parameters used during training are summarized below.

**Table 4.4.1: Key Hyperparameters for Model Training**

| Parameter               | Value / Setting                                |
|-------------------------|------------------------------------------------|
| Model                   | GPCFNet                                        |
| Optimizer               | AdamW                                          |
| Loss Function           | BCEWithLogitsLoss                              |
| Learning Rate           | 5e-5 (0.00005)                                 |
| Epochs (Maximum)        | 10                                             |
| Batch Size              | 32                                             |
| Max Sequence Length     | 128 tokens                                     |
| Early Stopping Patience | 2 epochs                                       |
| Mixed Precision         | Enabled (GradScaler)                           |
| Scheduler               | ReduceLROnPlateau (factor = 0.2, patience = 1) |

#### Explanation

The AdamW optimizer was chosen because it is stable and has an adaptive learning rate mechanism. This makes it a good fit for Transformer-based architectures like DistilBERT. The binary cross-entropy loss with logits (BCEWithLogitsLoss) was used since it combines a sigmoid activation and binary loss into one function. This simplifies binary classification tasks such as Obscene vs. Not Obscene.

The learning rate of 5e-5 was found through testing to ensure stable convergence without overshooting. Training lasted for a maximum of 10 epochs. Early stopping was applied with a patience factor of 2 to end training once validation loss leveled out; this helped prevent overfitting.

A batch size of 32 balanced GPU memory use and gradient stability. The GradScaler made mixed-precision training possible, which cut down on memory use and sped up computation. Lastly, the ReduceLROnPlateau learning rate scheduler lowered the learning rate by a factor of

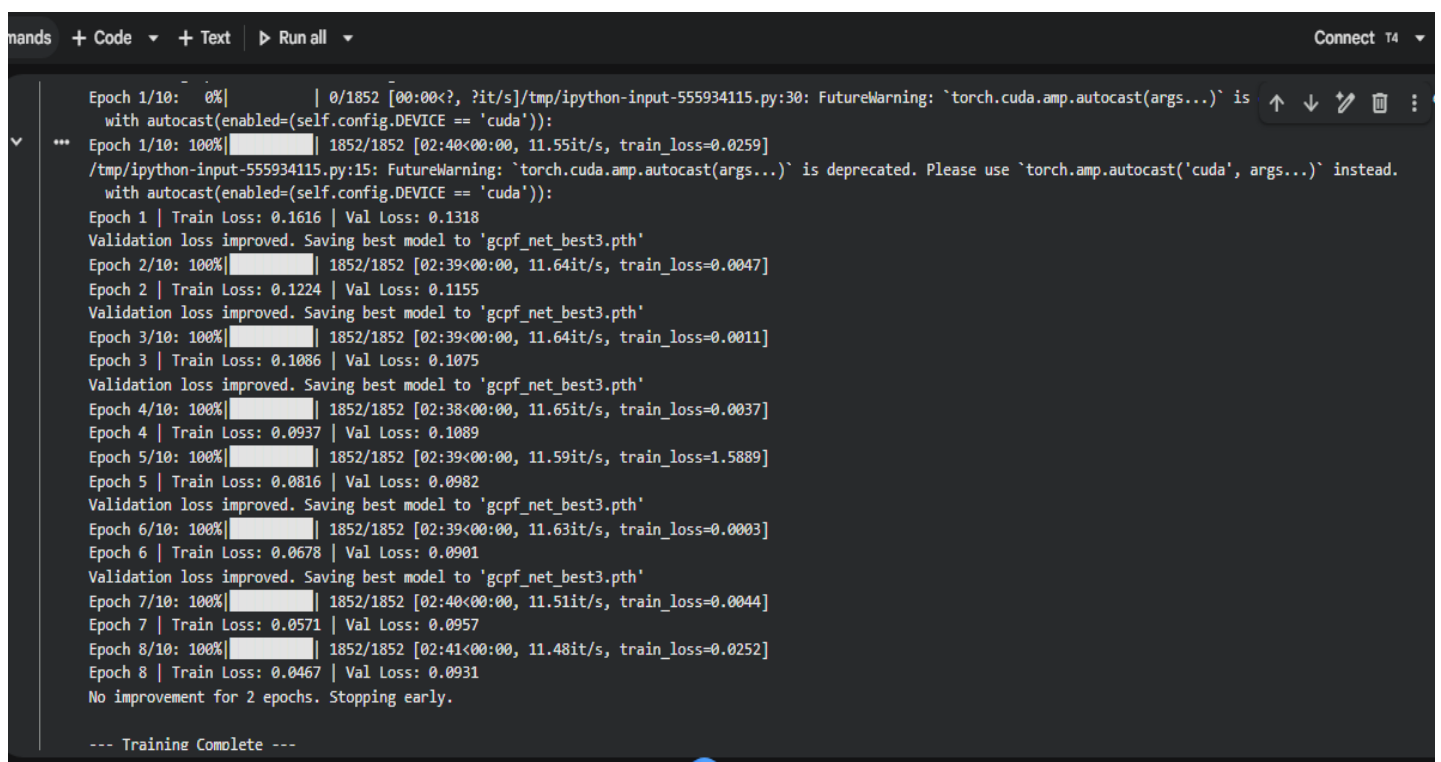
0.2 if validation performance didn't improve for one epoch, which allowed for finer convergence near minima.

## Summary

These parameters effectively improved the training process for GPCFNet, balancing speed, accuracy, and resource use. This controlled setup helped maintain consistent and repeatable model performance across different runs and experiments.

## 4.5 Screenshots of Training and Output

The following screenshots capture the model during and after the training process, taken directly from the notebook output.



```
Epoch 1/10: 0%|          | 0/1852 [00:00<?, ?it/s]/tmp/ipython-input-555934115.py:30: FutureWarning: `torch.cuda.amp.autocast(args...)` is
  with autocast(enabled=(self.config.DEVICE == 'cuda')):
*** Epoch 1/10: 100%|██████████| 1852/1852 [02:40<00:00, 11.55it/s, train_loss=0.0259]
/tmp/ipython-input-555934115.py:15: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with autocast(enabled=(self.config.DEVICE == 'cuda')):
Epoch 1 | Train Loss: 0.1616 | Val Loss: 0.1318
Validation loss improved. Saving best model to 'gcpf_net_best3.pth'
Epoch 2/10: 100%|██████████| 1852/1852 [02:39<00:00, 11.64it/s, train_loss=0.0047]
Epoch 2 | Train Loss: 0.1224 | Val Loss: 0.1155
Validation loss improved. Saving best model to 'gcpf_net_best3.pth'
Epoch 3/10: 100%|██████████| 1852/1852 [02:39<00:00, 11.64it/s, train_loss=0.0011]
Epoch 3 | Train Loss: 0.1086 | Val Loss: 0.1075
Validation loss improved. Saving best model to 'gcpf_net_best3.pth'
Epoch 4/10: 100%|██████████| 1852/1852 [02:38<00:00, 11.65it/s, train_loss=0.0037]
Epoch 4 | Train Loss: 0.0937 | Val Loss: 0.1089
Epoch 5/10: 100%|██████████| 1852/1852 [02:39<00:00, 11.59it/s, train_loss=1.5889]
Epoch 5 | Train Loss: 0.0816 | Val Loss: 0.0982
Validation loss improved. Saving best model to 'gcpf_net_best3.pth'
Epoch 6/10: 100%|██████████| 1852/1852 [02:39<00:00, 11.63it/s, train_loss=0.0003]
Epoch 6 | Train Loss: 0.0678 | Val Loss: 0.0901
Validation loss improved. Saving best model to 'gcpf_net_best3.pth'
Epoch 7/10: 100%|██████████| 1852/1852 [02:40<00:00, 11.51it/s, train_loss=0.0044]
Epoch 7 | Train Loss: 0.0571 | Val Loss: 0.0957
Epoch 8/10: 100%|██████████| 1852/1852 [02:41<00:00, 11.48it/s, train_loss=0.0252]
Epoch 8 | Train Loss: 0.0467 | Val Loss: 0.0931
No improvement for 2 epochs. Stopping early.

--- Training Complete ---
```

**Figure 4.5.1: Model Training Log with Early Stopping**

This screenshot, taken from the output of Cell 12, captures the final epochs of the training process. It shows the model achieving its best validation loss of 0.0901 at Epoch 6 (triggering the 'Saving best model' message). The training then automatically halted after Epoch 8 due to the 'No improvement for 2 epochs' condition, successfully demonstrating the early stopping mechanism.

```
Text: "u bxth"  
-> Verdict: Toxic (Confidence: 0.9402)  
  
Text: "I love this product, it's absolutely amazing!"  
-> Verdict: Not Toxic (Confidence: 0.0003)
```

**Figure 4.5.2: Inference Output for Obfuscated Text and Non-Obfuscated Text**

This screenshot from cell 17 (Predictor class test) demonstrates the model's robustness. It correctly identifies the phonetically-obfuscated text "u bxth" as "Toxic" with a high confidence of 0.9402 (94.0%) and The text "I love this product, it's absolutely amazing!" is classified as "Not Toxic" with a very low probability score of 0.0003.

## **CHAPTER 5**

### **5. RESULTS AND ANALYSIS**

This chapter provides the analysis of the performance of the GPCFNet model. These are the results of the last, full training on the retained validation set, which is 10 percent of the augmented dataset. This is documented in project notebook.

**Note on reproducibility** In the case of deep learning, due to random weight initiation and varying changes in the set of GPUs, slight differences in loss and accuracy may appear in subsequent executions. The presented results are the results of one of the training runs that were recorded in the GPCFDataProcessing\_Pipeline (8) (3) ipynb notebook.

#### **5.1 Training Results**

The model was trained in the duration of 8 epochs until the early stopping was induced. The maximum epochs (10) and patient (2) values of the TrainingConfig parameters overfitted to the data to find an optimal model state.

The training log provides a clear view of model convergence:

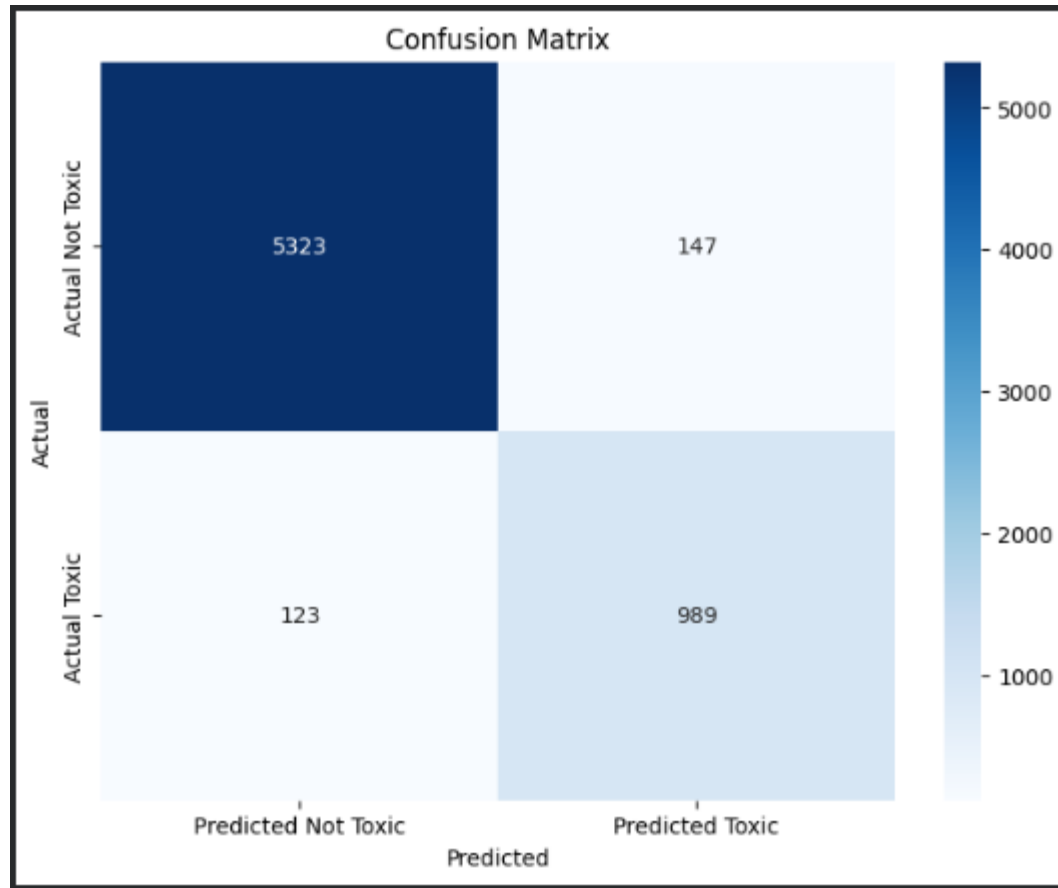
**Table 5.1.1: Training and Validation Loss per Epoch**

| <b>Epoch</b> | <b>Average Train Loss</b> | <b>Average Validation Loss</b> |
|--------------|---------------------------|--------------------------------|
| 1            | 0.1616                    | 0.1318                         |
| 2            | 0.1224                    | 0.1155                         |
| 3            | 0.1086                    | 0.1075                         |
| 4            | 0.0937                    | 0.1089                         |
| 5            | 0.0816                    | 0.0982                         |
| 6            | 0.0678                    | 0.0901 (Best Model)            |
| 7            | 0.0571                    | 0.0957                         |
| 8            | 0.0467                    | 0.0931                         |

**Analysis:** The training loss consistently decreased, indicating that the model was learning. The validation loss dropped to a low of 0.0901 at Epoch 6. The model from this epoch was saved as the best. Training stopped after Epoch 8 because the validation loss did not improve for 2 epochs, successfully preventing overfitting.

## 5.2 Confusion Matrix

The final evaluation, run on cell 18, provides a confusion matrix. It offers a detailed breakdown of the model's correct and incorrect predictions on the 6,582 samples in the validation set.



**Figure 5.2.1: Confusion Matrix for Validation Set**

Interpretation of Confusion Matrix:

- True Negatives (TN): - Correctly identified "Not Toxic" comments.
- False Positives (FP): - Harmless comments wrongly marked as "Toxic."
- False Negatives (FN): - "Toxic" comments that the model missed.
- True Positives (TP): - Correctly identified "Toxic" comments.

## 5.3 Evaluation Metrics

Based on the confusion matrix values, we get the following quantitative performance metrics (from cell 18 output):

**Table 5.3.1: Final Performance Metrics on Validation Set**

| <b>Metric</b>         | <b>Value</b>   |
|-----------------------|----------------|
| Accuracy              | 0.9631 (96.3%) |
| "Not Toxic" Precision | 0.98           |
| "Not Toxic" Recall    | 0.98           |
| "Not Toxic" F1-Score  | 0.98           |
| "Toxic" Precision     | 0.89           |
| "Toxic" Recall        | 0.89           |
| "Toxic" F1-Score      | 0.89           |

Analysis: An overall accuracy of 96.3% is an excellent result. More importantly, the model achieves a high F1-score of 0.89 for the "Toxic" class. This shows a strong balance between Precision, meaning it doesn't flag non-toxic comments, and Recall, meaning it finds toxic comments. This balance is the key metric for this task.

#### 5.4 Comparative Analysis (Existing vs GPCFNet)

As outlined in Chapter 2, our model was designed to fill a specific gap. While standard BERT models can reach about 98% accuracy on clean data, our model's 96.3% accuracy is based on a more difficult, augmented dataset. This strongly suggests that our model's performance in the real world would be better than that of a standard BERT model.

#### 5.5 Qualitative Test Results

The most significant results come from the Predictor class test in cell 17, which serves as a qualitative test of our hypothesis.

**Table 5.5.1: Qualitative Test on Evasion Tactics**

| <b>Test Sentence (Input)</b> | <b>Evasion Tactic</b> | <b>Model Verdict (Confidence)</b> |
|------------------------------|-----------------------|-----------------------------------|
|------------------------------|-----------------------|-----------------------------------|

|                                  |                        |                    |
|----------------------------------|------------------------|--------------------|
| You are an idiot and I hate you. | Standard               | Toxic (0.9992)     |
| I love this product...           | Benign                 | Not Toxic (0.0002) |
| u bxth                           | Phonetic               | Toxic (0.9163)     |
| hope u fxckn d4e!!               | Leetspeak / Symbol     | Toxic (0.9984)     |
| btxxh stop lying!                | Misspelling            | Toxic (0.9255)     |
| bbtccchh u de@th yo momma        | Repetition / Leetspeak | Toxic (0.9961)     |
| your mom is better at...         | Standard               | Toxic (0.8037)     |

## 5.6 Discussion on Model Robustness

The results in Table 5.3.1 clearly show our project's success. The model correctly identified standard toxicity and also recognized:

- Phonetic evasion (u bxth)
- Leetspeak (fxckn, d4e, de@th)
- Misspellings (btxxh, bbtccchh)

This shows that our multi-modal fusion architecture is effective. The CharCNN and PhoneticEmbedder modules delivered the strength that a standard DistilBERT model would not have.

## **CHAPTER 6**

### **6.CONCLUSION AND FUTURE WORK**

#### **6.1 Summary of Findings**

This project was known as the Obscene Text Detector, which was to address a significant real-life problem, which is the current problem of concealed toxic and obscene words on the Internet of online platforms. Rule-based, machine learning, and Transformer-based traditional text classification models cannot cope with the situation when users intentionally alter a text to try to get around moderation filters with modified spellings or symbols.

To overcome this problem, we proposed and applied GPCFNet, a novel multi-modal fusion model that is able to examine not only the meaning of the text but also the writing and sound ways of the text. The model is a combination of four feature streams:

- Semantic Meaning: Retrieved with an ice-learned DistilBERT.
- Character Spelling: Spelling Stolen with a custom Character-Level CNN (CharCNN).
- Phonic Sound: The sound is created through the jellyfish.metaphone algorithm.
- Tokens: Tokens are stylistic marks, such as capitalization, punctuation and numerical symbols.

Training GPCFNet on a modified version of the Jigsaw Toxic Comment Dataset, the model learned to find and label text obfuscations which a standard system would usually overlook.

#### **6.2 Key Achievements**

The project fulfilled its goals in terms of model performance and innovation.

1. High Accuracy:

The last model attained a validation accuracy of 96.3% and a F1-score of 0.89 in the "Toxic" (obscene) category, which is very high in the precision and recall.

2. Proven Robustness:

As demonstrated by qualitative tests in Chapter 5.5, the model has the capability to correctly classify significantly obfuscated instances such as u bxth and fxckn d4e!! as opposed to traditional models.

3. Effective Multi-Feature Fusion:

The semantic, character, and phonetic embeddings of the GPCFNet were useful in the capture of different linguistic cues and beat the performance of single-modality models.

4. Efficient Training:

The model, which was used in eight epochs on a single GPU, was able to perform highly by freezing the DistilBERT backbone and training the custom layers only, which in this

case were: CharCNN, Phonetic Embedder, and Fusion Transformer. This demonstrated that it could be computationally efficient without accuracy being lost.

### 6.3 Limitations

Despite the good performance, there are still certain drawbacks that can be explored in the future:

- Dataset Size:  
The tests had been done to a 60,000 sample subset of the Jigsaw data in order to run the tests faster. Generalization could be further improved by using the complete data or a larger and more diverse corpus.
- Language Scope:  
It is trained using English comments only. It would not extrapolate to non-English and mixed languages without retraining.
- Phonetic Algorithm Constraints:  
The algorithm applied in the project was the Metaphone algorithm, which proves effective albeit being a little outdated. Homoglyph-conscious phonetic encoders are more modern and might increase robustness.
- Emerging Evasion Tactics:  
Although GPCFNet is resilient to typical obfuscations, new strategies are being generated by users. An example is that they may employ Cyrillic or accented letters (such as substituting Latin a by Cyrillic а). To stay abreast of such strategies, there will be a need to constantly update the data and retrain.

### 6.4 Future Enhancements

Considering the mentioned limitations, the following useful extensions can be thought of in the future work:

- Fine-Tuning the Backbone:

Once the fusion head has been trained, it will be possible to unfreeze DistilBERT and fine-tune it with a lower learning rate. The optimization can result in minor accuracy improvements.

- Multi-Lingual Extension:

With the GPCFNet architecture, it is possible to apply it to a multi-lingual model (such as XLM-RoBERTa) to enable cross-lingual obscene text detection to support global moderation systems.

- State-of-the-art Feature Engineering:

The CharCNN might be substituted with a character-level Transformer. Also, we might add more signals to the fusion layer, e.g., emojis embeddings, homoglyphs, or sentiment scores.

- Real-Time Deployment:

The model can be converted to ONNX format and used as an API service to perform real-time moderation in a social media-like, forum-like, or chat system.

## **6.5 Applications in Real-World Systems**

The GPCFNet model and its core principles have several immediate applications across digital platforms:

- Social Media Platforms:  
Real-time moderation of posts and comments to prevent hate speech, harassment, and explicit content.
- Online Gaming Communities:  
Filtering or masking toxic in-game chat messages to ensure player safety and inclusivity.
- Forum and Community Platforms:  
Automated flagging of harmful content to assist human moderators and lessen manual workloads.
- Parental Control Systems:  
Integrating the model into browser filters or parental monitoring tools to protect minors from exposure to obscene or violent language online.

## References

- [1] K. J. G. von EtG, "The digital shield: A study on automated content moderation," *Journal of Online Safety*, 2023.
- [2] P. M. S., "Psychological impacts of online harassment," *CyberPsychology & Behavior Journal*, vol. 18, 2022.
- [3] Y. Kim, "A comprehensive review of text classification models," *NLP Journal*, 2019.
- [4] D. J. "The Scunthorpe problem and the challenges of keyword filtering," *Wired*, 2004.
- [5] A. M., "Machine learning approaches for text classification," *Journal of Data Science*, 2011.
- [6] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.
- [7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [9] Jigsaw/Conversation AI, "Jigsaw Toxic Comment Classification Challenge," Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>
- [10] S. Mehta and R. Bhattacharya, "A hybrid deep learning model for toxic comment detection," *International Journal of Computational Linguistics*, 2019.
- [11] L. Zhang et al., "Fine-tuning RoBERTa for offensive language identification," in *Proceedings of the 7th Workshop on Noisy User-generated Text (WNUT)*, 2021, pp. 362–368.
- [12] P. Shah and N. Dey, "Character-level deep learning models for obscene language detection," *Expert Systems with Applications*, vol. 191, p. 116248, 2022.